

RNGD 지원 op — 목록 재검증 + NPU 실제 실행 검증

furiosa.torch.db.SUPPORTED_ATEN_OPS 97 개를 직접 추출해 목록을 다시 확인하고, op 마다 최소 그래프를 만들어 RNGD 에서 컴파일 · 실행까지 돌려 검증했습니다.

한 줄 결론

‘지원 목록 (SUPPORTED)’ 은 컴파일러가 “받겠다”고 선언한 목록일 뿐, NPU 실행 보장이 아닙니다. 97 개 중 89 개는 실제로 돌고, 6 개는 특정 모양에서만, 2 개는 목록에 있어도 안 됩니다.

97 개 검증

89 실행 OK

6 조건부

2 불가

목록 오류 1

정정 ① — import 경로

~~from furiosa.torch.extension import SUPPORTED_ATEN_OPS~~

→ extension 모듈엔 없음 (hasattr=False). 올바른 경로:

from furiosa.torch.db import SUPPORTED_ATEN_OPS

정정 ② — 목록 내용 1 개

기존 정리본: ~~detach_copy~~ → 실제 목록: ~~copy_~~ (제자리 복사)

나머지 96 개는 정확히 일치. 양쪽 다 97 개라 개수로는 안 드러남.

op 목록 추출 → 그래프화 → 컴파일 (EDF) → NPU 실행 → CPU 대조

op 하나마다 그 op 만 쓰는 최소 모듈을 만들어 furiosa-llm 과 같은 분해 경로로 컴파일하고 rngd:3 에서 실행해 CPU 결과와 비교했습니다.

```
# 1) 지원 op 목록 (97 개) — 권위 있는 출처
from furiosa.torch.db import SUPPORTED_ATEN_OPS
len(SUPPORTED_ATEN_OPS) # = 97 (IMPORTABLE = 156)
# 정의 : aten_config.py → native_compiler.is_supported_aten()
```

```
# 2) op 1 개당 최소 그래프 → 컴파일 → NPU 실행
import torch, furiosa.torch # 순서 : torch 먼저
from furiosa.torch import CompileModule
from torch._decomp import core_aten_decompositions
TABLE = dict(core_aten_decompositions()) # = furiosa-llm 경로
```

```
ep = torch.export.export(Mod().eval(), (x,))
    .run_decompositions(TABLE)
cm = CompileModule.from_exported(ep) # ← EDF 컴파일
cm.to(torch.device("rngd", 3))
out = cm(x.to(dev), device=dev) # ← NPU 실행
# out vs Mod()(x) (CPU eager) 비교
```

판정 3 축

present 분해 후 그래프에 그 op 노드가 실제로 남았는가
compile from_exported 가 EDF 생성에 성공하는가
run NPU 결과가 CPU eager 와 일치하는가

검증 3 단계 + 교차검증

- ① op 단독 그래프
- ② 실패 시 sigmoid+add 실연산 그래프에 끼워서 재시험
- ③ dtype · 랭크 · 모양 · API 바뀌가며 흔들기
- ④ 의심 8 개를 독립 에이전트가 “되게 만들어보라” 반대검증

분류별 결과 — 16 개 카테고리 / 97 개 op

카테고리	개수	상태	대표 op / 비고
Math unary	11	10 OK · 1 불가	abs·cos·exp·log·sqrt·rsqrt·neg… / isnan 불가
Arithmetic binary	10	OK	add·sub·mul·div(.Scalar 는 .Tensor 로 정규화)·pow·clamp
Conv / Matmul	3	OK (감소정밀도)	convolution·mm·bmm — 상대오차 ~0.23%
Activation	6	OK	relu·leaky_relu·sigmoid·tanh·softmax·log_softmax
Comparison	14	OK	eq·ne·lt·le·gt·ge(.Scalar/Tensor)·maximum·minimum
Logical	3	OK	logical_and·logical_not·logical_xor
Bitwise	6	OK	bitwise_and·or·xor (.Scalar/Tensor)
Reduction	9	8 OK · 1 조건부	sum·mean·amax·max·argmax·any·var_mean·topk / cumsum 정수만
Pooling	3	2 OK · 1 조건부	avg_pool2d·adaptive_avg_pool2d(→mean) / max_pool2d_with_indices 조건부
Shape / View	16	15 OK · 1 조건부	view·permute·transpose·expand·squeeze·cat·slice… / slice_scatter 조건부
Split	2	OK	split_with_sizes · split_with_sizes_copy
Copy / Clone	4	OK	clone · copy · copy_ · _to_copy
Indexing	5	2 OK · 3 조건부	index_put·scatter OK / index·index_select·gather 조건부
Conditional	1	OK	where.self
Creation	3	OK	full · full_like · fill
Padding	1	불가	constant_pad_nd

근거: verify_round1_all97.py 가 97 개를 모두 export→compile→run. .Scalar 사칙연산 · _copy 뷰 변형 등은 export 시 동등 형태 (.Tensor · 일반 뷰) 로 정규화되어 그 형태로 검증됨 .

elementwise 는 CPU 와 사실상 동일 , matmul 계열만 ~0.23%

matmul(conv·mm·bmm) 은 정상 실행되지만 텐서엔진 감소정밀도라 상대오차가 ~0.23% 로 일정합니다 . 크기를 키워도 상대오차는 그대로고 절대오차만 커집니다 .

케이스	max 절대오차	상대오차 (L2)	코사인 유사도
sigmoid(x)*2 (elementwise)	2.4e-07	6.2e-08	1.0000000
mm 8×8	0.019	0.00232	0.9999976
mm 64×64	0.074	0.00233	0.9999973
mm 256×256	0.185	0.00235	0.9999979
conv2d 3×3	0.058	0.00232	0.9999974

읽는 법

matmul 계열의 ‘오차’는 틀린 게 아니라 NPU 텐서엔진의 정상적인 감소정밀도입니다 . 상대오차 ~0.23% 고정 · 코사인 0.99999+ 로 방향이 사실상 일치합니다 . 나머지 단항 / 이항 / 활성화 / 비교 /shape 연산은 CPU 와 거의 비트 단위로 같습니다 (상대오차 ~1e-7).

불가 2 개 · 조건부 6 개 — 되는 / 안 되는 조건 (실측)

op	판정	되는 조건 / 안 되는 조건	실측 근거
isnan	불가	float16/32/bf16·랭크 0~4·where/any/cast 등 11 가지 전부 컴파일 실패	(x != x) 로 바꾸면 컴파일 · 실행 OK → 막는 건 isnan 노드 하나
constant_pad_nd	불가	F.pad·1D/2D/3D pad·0/ 비 0 value·conv 뒤 ·Linear 앞 등 12 가지 전부 실패	주변 op(conv/sigmoid/addmm) 는 사는데 pad 노드 때문에 그래프 실패
cumsum	조건부	정수 (int32/int64) 입력만 OK · float 입력은 dim·랭크 불문 전부 실패	int64 1D/2D/3D CPU 와 bit-exact · float 18 케이스 실패
index_select	조건부	맨 안쪽 (마지막) 차원이 8 의 배수일 때만 OK	cols 8·16 OK / 3·5·7·15·17 FAIL (rows 4~32 무관)
index.Tensor (x[idx])	조건부	맨 안쪽 차원이 4 의 배수면 OK · x[:, idx] (비 - 선두축) 는 실패	cols 4·8·12·16 OK / 3·5·7 FAIL
gather	조건부	사실상 1 차원 (벡터) gather 만 OK · 다중행 rank-2 는 실패	1D OK / (8,C)·(R,8) 다중행 정렬 ·dim 불문 전부 FAIL
slice_scatter	조건부	해당 축 전체 덮어쓰기만 OK (= 결과가 src 와 동일 , 무의미) · 부분 덮기 실패	full-overwrite OK / start·end 부분 변경 전부 실패
max_pool2d_with_indices	조건부	indices(int64) 출력은 절대 불가 · values 만 써도 kernel≥2 는 거의 실패	ResNet stem(56×56,k3s2p1)·8×8·plain pool 실패

공통 : 전부 CompileModule.from_exported 에서 UnsupportedOpError("failed to compile the graph") 로 막힘 (NPU 실행 단계 전) . op 자체는 분해되지 않고 그래프에 남아 있음 → EDF 백엔드에 해당 op 의 코드 생성이 없거나 모양 제약이 있음 .

gather/index 는 ‘맨 안쪽 차원 정렬’에 따라 성패가 갈린다

같은 코드라도 텐서 모양만 바꾸면 컴파일이 되고 안 됩니다. 맨 안쪽 (feature) 차원 크기로 경계가 정확히 갈립니다.

index_select(x, 0, idx) · 안쪽 차원 (cols) 스왑 (rows=8)

rows 는 4~32 전부 OK(무관). cols 가 8의 배수일 때만 OK.



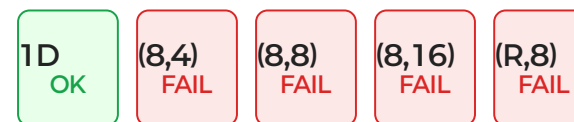
index.Tensor x[idx] · cols 스왑 (rows=6)

4의 배수면 OK.



gather · 차원 / 모양 스왑

사실상 1D(벡터)만 OK.



함의

임의의 모델을 그대로 올리면 gather·index·embedding-lookup·constant_pad_nd 의 feature 폭이 8(또는 4)의 배수가 아닐 때 컴파일이 막힐 수 있습니다. “지원 op” 라도 모양에 따라 안 됩니다. (참고: furiosa-smi status — 디바이스당 코어 8개, Core 0~7)